

PART TWO BUG REPORT

Restful Booker API Testing
Final Consolidated Report

PREPARED BY	DOCUMENT TYPE	STATUS
Jeremiah Igbiniedion	Final API Bug Report	Final

16 CONFIRMED DEFECTS | 4 TESTING PHASES | FINAL REPORT

Restful Booker API Testing

Prepared by: Jeremiah Igbiniedion
Document type: Final Part Two Bug Report
Execution date: 1 August 2026
Status: Final

1. Executive Summary

This report consolidates the confirmed findings from all Part Two testing phases:

- 1. Authentication and authorization testing.
- 2. CRUD and schema validation.
- 3. Negative and boundary testing.
- 4. Filtering testing.

Testing identified 16 confirmed API defects. The most significant risks relate to invalid financial data, silent type coercion, impossible date acceptance, incorrect HTTP status codes, weak authentication error handling and inaccurate date filtering.

Severity	Count
High	8
Medium	6
Low/Medium	1
Low	1
Total	16

2. Test Environment

Field	Value
Target	Local Restful Booker only
Base URL	http://127.0.0.1:3001
Docker image	mwinteringham/restfulbooker@sha256:e4a04233731ceca2e6d99bca16689e0d4bc44631047ab351b6f11b5fa08fb418
Framework	Playwright APIRequestContext with TypeScript and Zod
Execution mode	Serial, one worker

3. Scope and Test Approach

The following areas were tested:

- 1. Successful and unsuccessful token creation.
- 2. Required credential validation and malformed authentication bodies.
- 3. Valid, missing, invalid and stale token behaviour.
- 4. Authorization for PUT, PATCH and DELETE operations.
- 5. Create, read, full update, partial update and delete operations.
- 6. Response schema and persistence validation.
- 7. Invalid types, missing fields, invalid prices, invalid dates and malformed bodies.
- 8. Boundary values, long values, special characters and injection style input.
- 9. Name, date range and combined filtering.
- 10. Verification that rejected requests do not alter existing bookings.

11. Per test data creation, authenticated cleanup and deletion verification.

Every successful filtering response was parsed as a booking ID list. Returned IDs were retrieved individually and checked against the requested predicate. Every rejected PUT or PATCH request was followed by a GET request to confirm that the original booking remained unchanged. Invalid records unexpectedly accepted by POST were registered for authenticated teardown before the intended contract assertion.

4. Consolidated Test Results

4.1 Authentication, Authorization and Health

Scenario	Actual behaviour	Result
Valid credentials	200, unique 15 character hexadecimal token	Pass
Invalid username	200, Bad credentials, no token	BUG-API-001
Invalid password	200, Bad credentials, no token	BUG-API-001
Missing or empty username/password	200, generic Bad credentials	BUG-API-004
Empty JSON or zero byte auth body	200, generic Bad credentials	BUG-API-004
Malformed auth JSON	400 Bad Request, no token	Pass
Valid token on PUT	200, replacement persisted	Pass
Missing or invalid token on PUT	403, booking unchanged	Pass
Valid token on PATCH	200, selected fields changed and others preserved	Pass
Missing or invalid token on PATCH	403, booking unchanged	Pass
Valid token on DELETE	201, booking removed	Pass
Missing or invalid token on DELETE	403, booking remains available	Pass
Fabricated stale token	403, booking unchanged	Pass
GET /ping	201 Created with body Created	BUG-API-003

4.2 CRUD and Schema Validation

Scenario	Actual behaviour	Result
Create a unique valid booking	200, positive ID and complete booking returned	Pass
Read a created booking	200, response matches payload	Pass
Full authenticated update	200, replacement fields persisted	Pass
Partial authenticated update	200, selected fields changed and omitted fields preserved	Pass
Authenticated delete	201, subsequent GET returns 404	Pass
Complete CRUD lifecycle	State validated after every operation	Pass
Negative totalprice through POST, PUT and PATCH	Accepted and persisted	BUG-API-005
Blank names through POST, PUT and PATCH	Accepted and persisted	BUG-API-006
Missing mandatory POST field	500 Internal Server Error	BUG-API-007
String depositpaid	Accepted and converted to true	BUG-API-008
Checkout before check-in	Accepted and persisted	BUG-API-009
PUT, PATCH or DELETE of deleted ID	405 Method Not Allowed	BUG-API-010
Malformed JSON	400 Bad Request, no record created	Pass

4.3 Negative and Boundary Validation

Scenario	Actual behaviour	Result
Numeric firstname	POST and PUT return 500, PATCH persists the number	BUG-API-011
Boolean lastname	Accepted and persisted	BUG-API-011
String bookingdates	500 Internal Server Error	BUG-API-011
Numeric nested fields or additional needs	Accepted or inconsistently rejected	BUG-API-011
Decimal price 99.95	Stored as 99	BUG-API-012
Numeric and nonnumeric string prices	Converted to a number or null	BUG-API-012
Impossible calendar dates	Normalized and persisted	BUG-API-013
Zero byte or array POST body	500 Internal Server Error	BUG-API-014
Whitespace only names	Stored as empty strings	BUG-API-006 expansion
String "false" for deposit	Stored as Boolean true	BUG-API-008 expansion
Checkout equal to check-in	Accepted	BUG-API-009 expansion
Zero and maximum safe integer price	Preserved	Pass, product limits unspecified
5,000 character firstname	Preserved	Characterization, maximum unspecified
JSON null and malformed POST	400 Bad Request	Pass
Invalid PUT bodies	400, booking unchanged	Pass
Zero byte or array PATCH body	200, no mutation	Characterization
Unknown field	Ignored and not persisted	Pass
SQL, HTML, path and control strings	Stored as data with no cross record impact	Pass

4.4 Filtering

Scenario	Actual behaviour	Result
Exact firstname	Only exact case controlled records returned	Pass
Exact lastname	Only matching controlled records returned	Pass
Firstname and lastname	Only combined match returned	Pass
No match	200 with []	Pass
Apostrophe and ampersand	Correctly encoded and matched	Pass
Name case variants	Separate case sensitive result sets	Characterization
Check-in equal to lower boundary	Exact boundary booking excluded	BUG-API-015
Checkout upper boundary	Equal and earlier checkout included	Pass
Combined date range	Exact check-in booking excluded	BUG-API-015
Inverted range	200 with []	Pass
Invalid textual date	500 Internal Server Error	BUG-API-016
Impossible calendar date	Normalized and queried	BUG-API-013 extension
Unknown parameter	Ignored	Pass
limit=1 and offset=1	Full collection returned	Characterization

5. Confirmed Bug Reports

BUG-API-001, Invalid credentials return HTTP 200

Severity: Medium

Endpoint: POST /auth

Reproducibility: 100%

Preconditions: The Restful Booker API is running and the authentication endpoint is available.

Steps to reproduce:

1. Send POST /auth with an invalid username and valid password.
2. Repeat with a valid username and invalid password.
3. Inspect the HTTP status and response body.

Expected result: The API returns 401 Unauthorized, a structured authentication error and no token.

Actual result: The API returns 200 OK with { "reason": "Bad credentials" } and no token.

Impact: Clients, monitoring systems and security telemetry may treat failed authentication as successful HTTP traffic.

Automated coverage: tests/authentication.spec.ts

BUG-API-002, Tokens have no expiration or revocation mechanism

Severity: Medium in the disposable test environment, potentially High in production

Endpoints: POST /auth and protected booking operations

Reproducibility: 100% while the process remains active

Preconditions: A valid token has been created.

Steps to reproduce:

1. Authenticate and obtain a valid token.
2. Continue using the token on protected operations while the process remains active.
3. Review the available token lifecycle controls.
4. Restart the container and retry the previous token.

Expected result: Tokens have a documented lifetime, deterministic expiration and a supported revocation or logout mechanism.

Actual result: Tokens are stored in process memory without timestamps, TTL, logout, revocation, scheduled cleanup or deletion logic. They remain usable while the process runs. A container restart invalidates them only because the in-memory state is lost.

Impact: A compromised token can remain usable indefinitely during process uptime.

Limitation: Container restart proves stale token rejection, not natural time based expiration.

Automated coverage: tests/authorization.spec.ts covers deterministic invalid and stale token rejection. No arbitrary timed expected failure is included.

BUG-API-003, GET /ping returns HTTP 201 Created

Severity: Low

Endpoint: GET /ping

Reproducibility: 100%

Steps to reproduce:

1. Send GET /ping.
2. Inspect the status code and body.

Expected result: 200 OK is returned for a successful health read.

Actual result: 201 Created is returned with body Created, although no resource is created.

Impact: Health checks and monitoring receive semantically incorrect status information.

Automated coverage: tests/health.spec.ts

BUG-API-004, Required credentials receive generic authentication errors

Severity: Low/Medium

Endpoint: POST /auth

Reproducibility: 100%

Steps to reproduce:

1. Omit username from an otherwise valid authentication request.
2. Repeat by omitting password.
3. Repeat with empty username, empty password, {} and a zero byte body.
4. Inspect each response.

Expected result: 400 Bad Request identifies the missing or empty credential field.

Actual result: 200 OK returns the generic response { "reason": "Bad credentials" }.

Impact: Consumers cannot distinguish structurally incomplete requests from invalid credentials or give precise corrective feedback.

Automated coverage: tests/authentication.spec.ts

BUG-API-005, Booking mutations accept negative total prices

Severity: High

Endpoints: POST /booking, PUT /booking/{id}, PATCH /booking/{id}

Reproducibility: 100% across three runs

Steps to reproduce:

1. Submit an otherwise valid booking with "totalprice": -1.
2. Repeat through authenticated PUT and PATCH requests.
3. Retrieve the affected booking.

Expected result: 400 Bad Request explains that price must be nonnegative. An existing booking remains unchanged.

Actual result: 200 OK is returned and the negative price is persisted.

Impact: Invalid financial data can affect billing, reconciliation and reporting.

Automated coverage: tests/schema-validation.spec.ts

BUG-API-006, Booking mutations accept blank and whitespace only customer names

Severity: Medium

Endpoints: POST /booking, PUT /booking/{id}, PATCH /booking/{id}

Reproducibility: 100% across three runs

Steps to reproduce:

1. Submit an otherwise valid booking with empty firstname and lastname values.
2. Repeat with whitespace only values.
3. Repeat through PUT and PATCH.
4. Retrieve the booking.

Expected result: 400 Bad Request returns field specific validation and existing data remains unchanged.

Actual result: Empty values are persisted. Whitespace only values are trimmed and stored as empty strings.

Impact: Bookings can exist without usable customer identity information.

Automated coverage: tests/schema-validation.spec.ts and tests/negative-boundary.spec.ts

BUG-API-007, Missing mandatory booking fields produce HTTP 500

Severity: High

Endpoint: POST /booking

Reproducibility: 100% for firstname, lastname, totalprice, depositpaid and bookingdates

Steps to reproduce:

1. Remove one mandatory field from an otherwise valid POST body.
2. Submit the request.
3. Repeat for each mandatory field.

Expected result: 400 Bad Request returns a structured message naming the missing field.

Actual result: 500 Internal Server Error is returned with a generic text body.

Impact: Client mistakes are misreported as server failures, obscuring remediation and polluting monitoring.

Automated coverage: tests/schema-validation.spec.ts

BUG-API-008, Non-boolean deposit status is accepted and coerced

Severity: High

Endpoints: POST /booking, PUT /booking/{id}, PATCH /booking/{id}

Reproducibility: 100% across three runs

Steps to reproduce:

1. Submit valid JSON containing "depositpaid": "ddd".
2. Retrieve the booking.
3. Repeat with "depositpaid": "false" through POST, PUT and PATCH.

Expected result: 400 Bad Request is returned because only JSON Boolean true or false is valid. Existing data remains unchanged.

Actual result: 200 OK is returned and both strings are silently stored as Boolean true.

Impact: Invalid input can incorrectly mark a deposit as paid, creating financial and data integrity risk.

Automated coverage: tests/schema-validation.spec.ts and tests/negative-boundary.spec.ts

BUG-API-009, Invalid booking date order is accepted

Severity: High

Endpoints: POST /booking, PUT /booking/{id}, PATCH /booking/{id}

Reproducibility: 100% across three runs

Steps to reproduce:

1. Submit check-in 2026-11-20 and checkout 2026-11-10.
2. Retrieve the booking.
3. Repeat with checkout equal to check-in.

Expected result: 400 Bad Request explains that checkout must be later than check-in and existing data remains unchanged.

Actual result: Reversed date ranges and equal dates are accepted and persisted.

Impact: Impossible or zero night stays can corrupt availability, duration, pricing and reporting logic.

Automated coverage: tests/schema-validation.spec.ts and tests/negative-boundary.spec.ts

BUG-API-010, Operations on nonexistent bookings return HTTP 405

Severity: Medium

Endpoints: PUT /booking/{id}, PATCH /booking/{id}, DELETE /booking/{id}

Reproducibility: 100% across three runs

Steps to reproduce:

1. Create and delete a booking.
2. Confirm GET for the ID returns 404.
3. Perform authenticated PUT, PATCH and DELETE requests on the deleted ID.

Expected result: Each request returns 404 Not Found because the resource does not exist.

Actual result: Each request returns 405 Method Not Allowed, although the methods are supported for existing bookings.

Impact: Clients receive the wrong failure category and may apply incorrect recovery behaviour.

Automated coverage: tests/schema-validation.spec.ts

BUG-API-011, Booking field types are inconsistently validated

Severity: High

Endpoints: POST, PUT and PATCH /booking

Reproducibility: 100%

Steps to reproduce:

1. Submit a numeric firstname, Boolean lastname, string bookingdates or numeric nested fields.
2. Repeat through POST, PUT and PATCH.
3. Inspect the response and persisted booking.

Expected result: Wrong field types return 400 Bad Request with field specific validation and do not mutate data.

Actual result: Wrong types are variously persisted, coerced or reported as 500, depending on the field and HTTP method.

Impact: Invalid response schemas reach clients and client errors are misclassified as server failures.

Automated coverage: tests/negative-boundary.spec.ts

BUG-API-012, Price coercion causes silent data corruption

Severity: High

Endpoints: POST, PUT and PATCH /booking

Reproducibility: 100%

Steps to reproduce:

1. Submit "totalprice": 99.95.
2. Repeat with "totalprice": "111" and "totalprice": "eleven".
3. Retrieve each booking.

Expected result: Decimal precision is preserved when decimals are supported. String prices return 400 Bad Request.

Actual result: 99.95 becomes 99, "111" becomes 111 and "eleven" becomes null.

Impact: Billing and reconciliation can use values the client did not submit.

Automated coverage: tests/negative-boundary.spec.ts

BUG-API-013, Impossible calendar dates are silently normalized

Severity: High

Endpoints: POST, PUT and PATCH /booking, plus booking date filters

Reproducibility: 100%

Steps to reproduce:

1. Submit a booking containing an impossible date such as 2031-02-30.
2. Retrieve the booking.
3. Send a booking filter containing the same impossible date.

Expected result: 400 Bad Request returns a date validation error.

Actual result: The date is shifted into March and accepted or used as a filter.

Impact: The system creates or returns bookings for dates the customer did not request.

Automated coverage: tests/negative-boundary.spec.ts and tests/filtering.spec.ts

BUG-API-014, Empty and array POST bodies cause internal server errors

Severity: High

Endpoint: POST /booking

Reproducibility: 100%

Steps to reproduce:

1. Send POST /booking with a zero byte body.
2. Repeat with a JSON array body.
3. Inspect each response.

Expected result: 400 Bad Request explains that a booking object is required.

Actual result: Both requests return 500 Internal Server Error.

Impact: Malformed client input appears as a service failure and pollutes monitoring.

Automated coverage: tests/negative-boundary.spec.ts

BUG-API-015, Check-in filtering excludes the documented lower boundary

Severity: Medium

Endpoint: GET /booking?checkin={date}

Reproducibility: 100%

Preconditions: Create one booking on 2032-04-10 and another on 2032-04-11.

Steps to reproduce:

1. Filter bookings using checkin=2032-04-10.
2. Repeat as part of a combined date range.
3. Inspect the returned booking IDs.

Expected result: Both IDs are returned because the embedded API documentation specifies greater than or equal behaviour.

Actual result: The exact boundary booking is excluded and only the later booking is returned.

Impact: Valid bookings disappear from exact boundary and combined date range searches.

Automated coverage: tests/filtering.spec.ts

BUG-API-016, Invalid filtering dates produce HTTP 500

Severity: Medium

Endpoint: GET /booking?checkin={date}

Reproducibility: 100%

Steps to reproduce:

- 1. Send GET /booking?checkin=not-a-date.
- 2. Inspect the response.

Expected result: 400 Bad Request returns a date validation error.

Actual result: 500 Internal Server Error is returned.

Impact: Consumer input errors appear as service outages.

Automated coverage: tests/filtering.spec.ts

6. Repeatability and Cleanup Evidence

The complete local suite was executed three consecutive times without resetting or manually preparing test data.

Run	Total tests	Normal passes	Expected known defect failures	Unexpected failures	Result
1	101	43	58	0	Passed
2	101	43	58	0	Passed
3	101	43	58	0	Passed

The booking collection contained 25 records before execution and returned to 25 afterward. Each protected endpoint test created a separate run scoped booking. Fixture teardown authenticated every deletion and confirmed that every tracked ID returned 404. Unauthorized requests were followed by GET assertions proving that the original booking remained unchanged.

7. Expected Failure Policy

Confirmed deterministic defects remain executable using Playwright's `test.fail` marker. Each test asserts the intended correct behaviour instead of accepting the defective response.

- 1. A documented defect that fails as expected is reported as an expected failure and does not keep CI permanently red.
- 2. A test marked as an expected failure that unexpectedly passes fails the Playwright run, prompting defect retesting and removal of the marker when fixed.
- 3. New regressions that are not explicitly marked remain ordinary failures.
- 4. Characterization tests and unresolved product questions are not marked as expected failures.
- 5. Assertions confirming that failed authentication creates no token run before the known defect status assertion.

8. Product Questions and Limitations

8.1 Duplicate Dates and Overbooking

Two different controlled customers with identical check-in and checkout dates were accepted with separate booking IDs. This is not classified as a defect because the API contains no room, property, capacity or inventory identifier.

Clarification is required on:

1. The resource or inventory unit reserved by a booking.
2. How capacity is represented when no room or property identifier exists.
3. Whether all overlapping dates should be rejected or only overlaps for the same resource.
4. The expected status and error contract for an availability conflict.

8.2 Additional Product Questions

1. Name filtering is case sensitive. Case insensitive search may improve usability, but it is not required by the current contract.
2. `limit` and `offset` are ignored. Pagination is a scalability limitation, but it is not promised by the published contract.
3. The embedded checkout documentation describes greater than or equal behaviour, while the observed implementation behaves as an inclusive upper bound. The intended contract requires clarification.
4. Price maximums, free bookings and name length limits are unspecified.
5. Empty or array PATCH bodies behave as no-op requests. Whether they should be rejected is unspecified.
6. Stored HTML remains a downstream rendering risk, but no executable XSS was proven at the API layer.

9. Confirmed Non-Defects and Observations

1. Malformed JSON returns 400 Bad Request and creates no residual booking.
2. JSON null POST bodies return 400 Bad Request.
3. Invalid PUT bodies return 400 and preserve the original booking.
4. Unknown booking fields are ignored and not persisted.
5. SQL, HTML, path and control strings remained JSON data and caused no cross record impact.
6. Exact and combined name filters return the correct controlled data.
7. No match filters return 200 with an empty array.
8. Apostrophes and ampersands are encoded and matched correctly.
9. Inverted date ranges return no matches.
10. Missing or invalid authorization tokens correctly return 403 and do not mutate protected resources.
11. GET /booking without authentication is documented behaviour and is not classified as an authorization defect.
12. POST returning 200 and successful DELETE returning 201 match the published Restful Booker contract, so they are not treated as functional defects in this report.

10. Conclusion

Part Two testing confirmed that the main CRUD workflow operates successfully for valid data, protected mutations reject missing or invalid tokens, filtering works for standard exact match scenarios and test data cleanup is reliable. However, the API has material validation and error handling weaknesses. High priority remediation should focus on financial field validation, strict type enforcement, date validation, mandatory field handling and prevention of silent coercion. Medium priority remediation should address authentication status semantics, nonexistent resource status codes and date filtering accuracy.